

Okres (2) – (3) Korekta aplikacji i przesyłanie konfiguracji

Poprawiona funkcjonalność

```
#pragma once
#include <cmath>

class RegulatorPID
{
public:
    enum LiczCalk { PROSTOKATNY, Wew, Zew }; //tryb metody obliczania całki

private:
    double Kp; //wzmocnienie regulatora
    double Ti; //stała całkowania
    double Td; //stała różniczkowania
    double T; //okres próbkowania
    double umin; //ograniczenia sygnału sterującego
    double umax;
    bool ograniczenia; //czy ograniczenia włączone
    double uchyb_poprzedni; //uchyb z poprzedniej iteracji
    double akum_wew;
    double akum_zew;
    LiczCalk typCalki=LiczCalk::Zew; //dodana domyslna
    static constexpr double EPS = 1e-12; //dzielenie przez 0
    ...
};
```

```
RegulatorPID(double Kp_ = 1.0, double Ti_ = 0.0, double Td_ = 0.0, double T_ = 1.0)
: Kp(Kp_), Ti(Ti_), Td(Td_), T(T_),
  umin(-10.0), umax(10.0),
  ograniczenia(true),
  uchyb_poprzedni(0.0), akum_wew(0.0), akum_zew(0.0),
  typCalki(PROSTOKATNY), P(0.0), I(0.0), D(0.0)
{}
```

```
ustawARXDane(
    aktualnyWektorA,
    aktualnyWektorB,
    aktualneOpoznienie,
    aktualnySzum,
    arx_uMin,
    arx_uMax,
    arx_yMin,
    arx_yMax,
    arx_ograniczenia
);
```

```

void MainWindow::on_Wczytaj_Button_clicked()
{
    QString sciezka = QFileDialog::getOpenFileName(this, "Wczytaj konfigurację", "", "JSON (*.json)");
    if (sciezka.isEmpty()) return;

    std::vector<double> a, b;
    int opoznienie;
    double odchylenie, Kp, Ti, Td;
    int typCalki, trybGeneratora;
    double amplituda, StalaSkadniowa, Wypelnienie, czestotliwosc;
    int interwalMs;

    bool sukces = menedzerKonfig.wczytajKonfiguracje(
        sciezka, a, b, opoznienie, odchylenie, arx_uMin, arx_uMax, arx_yMin, arx_yMax, Kp, Ti, Td, typCalki,
        trybGeneratora, amplituda, czestotliwosc, StalaSkadniowa, Wypelnienie, interwalMs
    );

    if (sukces) {
        QString sciezka = QFileDialog::getOpenFileName(this, "Wczytaj konfigurację", "", "JSON (*.json)");
        if (sciezka.isEmpty()) return;

        std::vector<double> a, b;
        int opoznienie;
        double odchylenie, Kp, Ti, Td;
        int typCalki, trybGeneratora;
        double amplituda, StalaSkadniowa, Wypelnienie, czestotliwosc;
        int interwalMs;
        double uMin, uMax, yMin, yMax;

        bool sukces = menedzerKonfig.wczytajKonfiguracje(
            sciezka, a, b, opoznienie, odchylenie,
            uMin, uMax, yMin, yMax,
            Kp, Ti, Td, typCalki,
            trybGeneratora, amplituda, czestotliwosc,
            StalaSkadniowa, Wypelnienie, interwalMs
        );

        if (sukces) {  }
    } else {
        QMessageBox::warning(this, "bład", "nie udało się wczytać");
    }
}

```

```

void MainWindow::on_Wczytaj_Button_clicked()
{
    QString sciezka = QFileDialog::getOpenFileName(this,
        "Wczytaj konfigurację",
        "",
        "JSON (*.json)");

    if (sciezka.isEmpty())
        return;

    std::vector<double> a, b;
    int opoznienie;
    double odchylenie, Kp, Ti, Td;
    int typCalki, trybGeneratora;
    double amplituda, StalaSkadniowa, Wypelnienie, czestotliwosc;
    int interwalMs;
    double uMin, uMax, yMin, yMax;

    bool sukces = menedzerKonfig.wczytajKonfiguracje(
        sciezka, a, b, opoznienie, odchylenie,
        uMin, uMax, yMin, yMax,
        Kp, Ti, Td, typCalki, trybGeneratora,
        amplituda, czestotliwosc, StalaSkadniowa, Wypelnienie,
        interwalMs);

    if (sukces) {  }
    else {
        QMessageBox::warning(this, "bład", "nie udało się wczytać");
    }
}

```







